

Algorithms

Damir Krstinić
damir.krstinic@fesb.hr

About...

- Knowledge
- Problem solving
- Computing tools
- Algorithm
- Turing machine
- Complexity

What is a knowledge?

Declarative knowledge

- ♦ Familiarity, awareness or understanding about someone or something gained through experience or study.

The sum of all that has been perceived, discovered or learned.

Declarative knowledge

- ♦ Familiarity, awareness or understanding about someone or something gained through experience or study.

The sum of all that has been perceived, discovered or learned:

- ♦ “Apple is delightful”

Declarative knowledge

- ♦ Familiarity, awareness or understanding about someone or something gained through experience or study.

The sum of all that has been perceived, discovered or learned:

- ♦ “Apple is delightful”
- ♦ “y is the square root of x only if $y*y = x$ ”

Declarative knowledge

- ♦ Familiarity, awareness or understanding about someone or something gained through experience or study.

The sum of all that has been perceived, discovered or learned:

- ♦ “Apple is delightful”
- ♦ “ y is the square root of x only if $y*y = x$ ”
- ♦ “A good health care plan improves the quality of medical care while saving money”

Imperative knowledge

- ◆ Imperative (procedural, implicit) knowledge is the knowledge excersised in the accomplishment of a task
- ◆ It is about how to accomplish something (a recipe)
- ◆ It can be directly applied to a task

Unlike the declarative knowledge, imperative knowledge cannot be easily articulated by the individual

- ◆ Most of the imperative knowledge is adopted without even being aware of the learning

Knowledge

- ◆ Knowledge is a familiarity with someone or something, which can include facts, information, descriptions, or skills acquired through experience or education.
- ◆ It can refer to the theoretical or practical understanding of a subject.
- ◆ It can be implicit (as with practical skill or expertise) or explicit (as with the theoretical understanding of a subject)

Problem solving

- ♦ Considered the most complex of all intellectual functions, higher-order cognitive process that requires the modulation and control of more routine or fundamental skills

Mathematical and personal problem solving

- ♦ Given some initial condition (input data), taking a set of steps to achieve desired final condition (output information)
 - ♦ It is about how to accomplish something (a recipe) – **an algorithm**

Mathematical problem solving

- ♦ Can be assisted by using computing tools

Does usage of more advance tool make problem solving easier or more complex?

Mathematical problem solving

- ♦ Can be assisted by using computing tools

Does usage of more advance tool make problem solving simpler or more complex?

- ♦ EXAMPLE: transfer from one place to another:
 - ♦ By foot
 - ♦ By car
 - ♦ By plane

Mathematical problem solving

- ♦ Can be assisted by using computing tools

Does usage of more advance tool make problem solving easier or more complex?

- ♦ EXAMPLE: transfer from one place to another:
 - ♦ By foot - hard way
 - ♦ By car - easier and faster
 - ♦ By plane - fastest

Mathematical problem solving

- ♦ Can be assisted by using computing tools

Does usage of more advance tool make problem solving easier or more complex?

- ♦ EXAMPLE: transfer from one place to another:
 - ♦ By foot - hard way, requires minimal knowledge
 - ♦ By car - easier and faster, requires advance skills
 - ♦ By plane - fastest, requires most advance and rare skill of flaying the plane!

Computing tools

- Sticks and stones

Computing tools

- ♦ Sticks and stones
- ♦ Paper and pencil

Computing tools

- Sticks and stones
- Paper and pencil
- Abacus

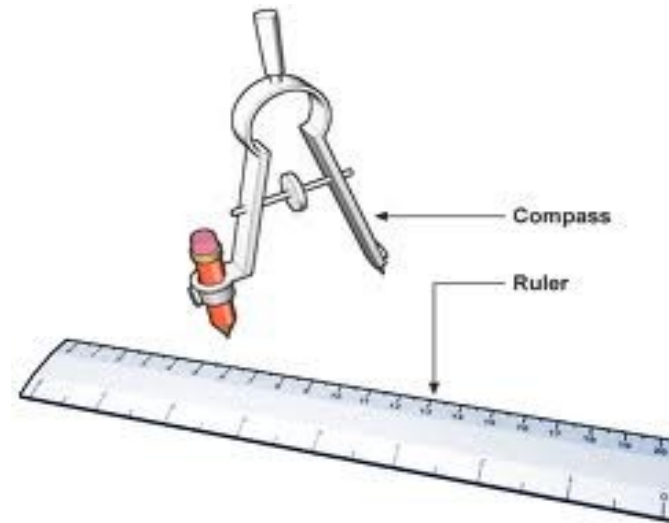
Computing tools

- Sticks and stones
- Paper and pencil
- Abacus
- Slide rules



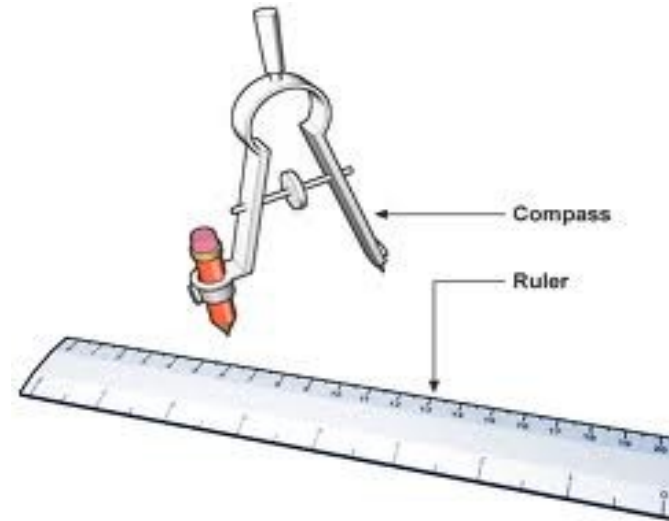
Computing tools

- Sticks and stones
- Paper and pencil
- Abacus
- Slide rules
- Ruler and compass



Computing tools

- ◆ Sticks and stones
- ◆ Paper and pencil
- ◆ Abacus
- ◆ Slide rules
- ◆ Ruler and compass
- ◆ ...
- ◆ Digital computer



Computing tools

- ♦ Each tool has set of capabilities called *primitive operations* or *primitives*
- ♦ The primitives may be combined in various ways to perform a computation

EXAMPLE: Ruler and compass

- ♦ **Ruler:** can draw lines and specify lengths
- ♦ **Compass:** can define arcs and circles

TASK 1:

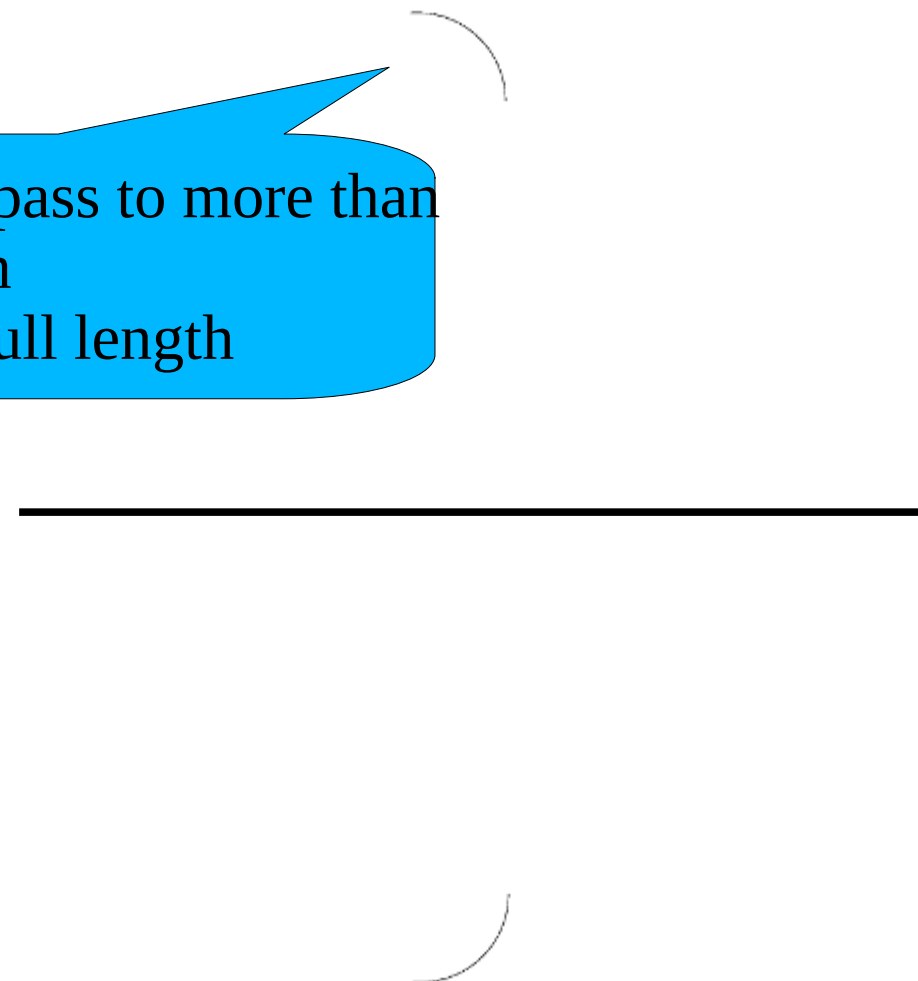
Divide number by 2 using ruler and compass

TASK 1: Divide number by 2 using ruler and compass

STEP 1: draw a line of a specific length



TASK 1: Divide number by 2 using ruler and compass



STEP 2: set compass to more than a half of a length and less than a full length

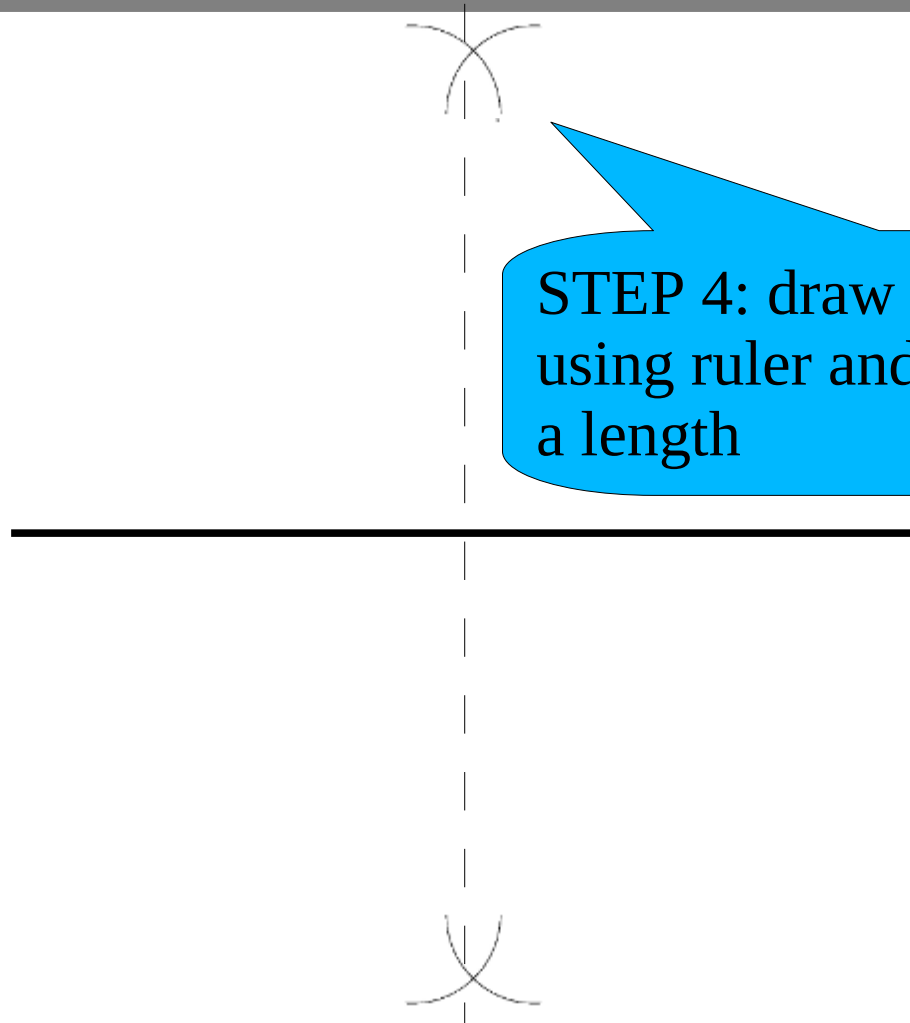
TASK 1: Divide number by 2 using ruller and compass



STEP 3: draw arcs from both
Ends of the line



TASK 1: Divide number by 2 using ruler and compass



STEP 4: draw a vertical line using ruler and measure half of a length

Algorithm

- Given a *problem* to be solved and a *tool* with its *set of primitive operations*, the goal is to evolve a combination of primitives of the tool in a certain order to solve the problem.

The algorithm is a finite specification of the solution of the problem given as an explicit combination of the primitives of the given tool in the specific order.

Algorithm

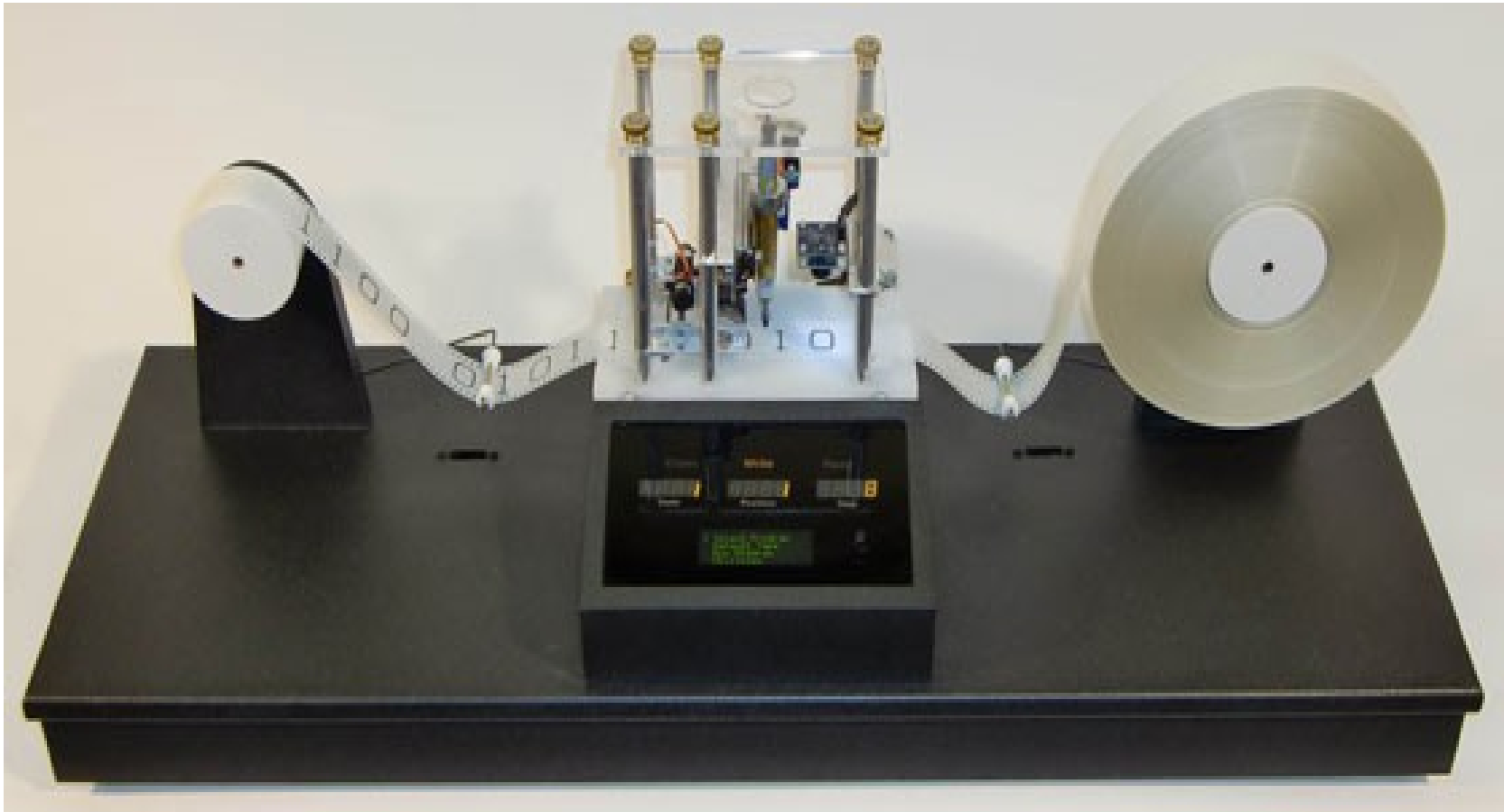
- ♦ **FINITE:** the number of steps in the computation is finite!
- ♦ **DEFINITE:** precisely specified set of a input and output values
- ♦ **UNAMBIGUOUS:** distinct set and order of operations (given in the language understandable by the tool!)
- ♦ **COMPLEXITY:** measure of the requirements for the algorithm execution (memory, time, ...)

Turing machine

- ♦ Hypothetical device that mechanically manipulates symbols on tape according to a table of rules
 - ♦ Operation (computation) is fully determined by a finite set of elementary instructions such as:

“In state 9, if the symbol on tape is 3, write symbol 10 on tape and change to state 42”

Turing machine



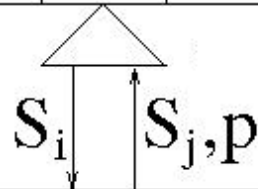
Turing machine - model

$$\mathbf{TS} = (\mathbf{Q}, \mathbf{S}, \delta, \mathbf{q}_0, \mathbf{h})$$

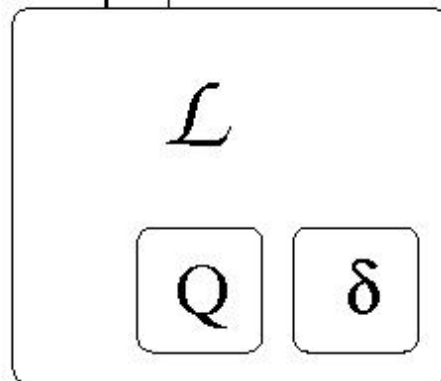
$\mathbf{Q} = \{\mathbf{q}_0, \mathbf{q}_1, \dots, \mathbf{q}_n, \mathbf{h}\}$	Finite set of internal states
$\mathbf{S} = \{\mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_m, \mathbf{B}\}$	Alfabet, B (Blank): empty symbol
δ	Transition function (table)
$\mathbf{q}_0 \in \mathbf{Q}$	Initial state
$\mathbf{h} \in \mathbf{Q}$	Halting state

Turing machine - model

Endless tape - external memory



Head - can read and write on tape,
move tape one cell left or right



$\mathcal{L}$ - logic unit

Q - internal memory of

δ - transition function, logic
function of the machine

S_i - input symbol (read from tape)

S_j - output symbol (written on tape)

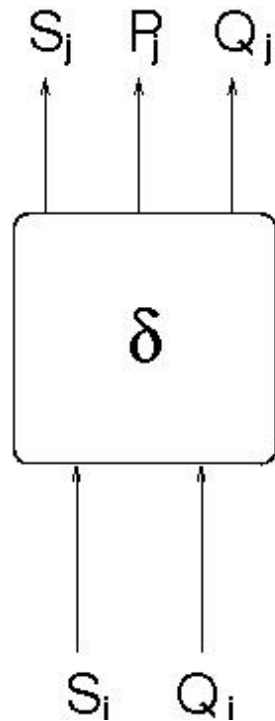
p - move (left, right, stay in the same position)

Turing machine

- ♦ Reads input information A (written as a sequence of symbols on the tape)
- ♦ Each step:
 - ♦ The machine reads symbol below the head
 - ♦ Based on symbol and internal state, logical unit generates output symbol, new state and move command
 - ♦ If halting state is reached, machine stops
- ♦ Output information B is written as a sequence of symbols on the tape

Turing machine

♦ Logic function:



S_i - input symbol read from tape in cycle i

Q_i - internal state in cycle i

S_i - output symbol (to be written on tape)

R_i - move direction (L, R, N - none)

Q_i - new internal state

Turing machine

- ♦ Logic function:

Mapping combination of the internal state and the symbol read from tape to a new internal state, move and symbol to be written on tape

$$(S_j, P_j, Q_j) = \delta(S_i, Q_i)$$

- ♦ If new state $Q_j = h$ (**HALT**) machine stops

Turing machine – logic function

	q0	q1	q2	...	h
s1	q1, s2, D	q3, s2, N	...	...	...
s2	q1, s1, D	q1, s2, L	...	...	...
s3	q2, s3, L	...	h, s3, N	...	...
...	...	...	...	...	...

- Logic function δ defines mapping

$$\delta: (S_i, Q_i) \rightarrow (S_j, P, Q_j)$$

- Logic function can be shown in the form of the transition table

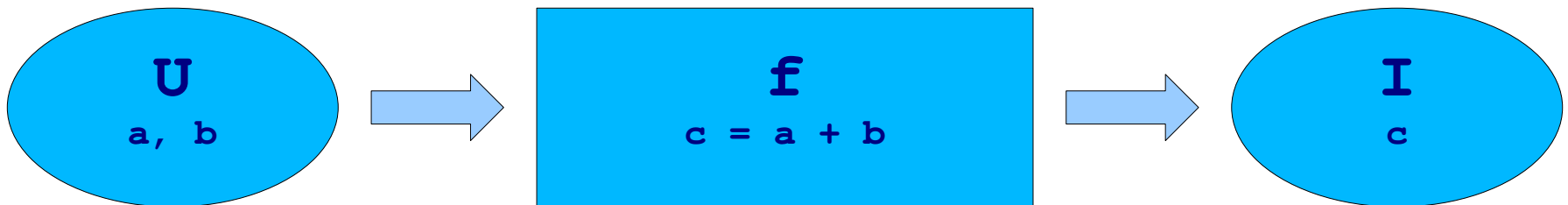
Turing machine - limitations

- ♦ What are the limitations of the Touring machine?
 - ♦ What set of problems could be solved with the Touring machine and is there any problem that cannot be solved with it?

Turing machine - limitations

- ♦ What are the limitations of the Turing machine?
 - ♦ Every algorithm could be represented in the form of mapping input values to output values (regardless of the type of data and input method)

EXAMPLE: add two numbers



Turing machine - limitations

- ♦ What are the limitations of the Turing machine?
 - ♦ Every algorithm could be represented in the form of mapping input values to output values (regardless of the type of data and input method)

Church – Turing thesis:

Function is algorithmically computable if and only if it is computable by a Turing machine.

Turing machine - limitations

- ♦ What are the limitations of the Turing machine?
 - ♦ What set of problems could be solved with the Turing machine and is there any problem that cannot be solved with it?

Despite its simplicity, a Turing machine can be adapted to simulate the logic of any computer algorithm!

Turing machine and digital computers

There is no problem that can be solved
with the most advance modern
computer that cannot be solved with
the Touring machine!

Algorithm

- The algorithm is a finite specification of the solution of the problem given as an explicit combination of the primitives of the given tool in the specific order.
- What is the algorithm of the Turing machine?

Algorithm

- The algorithm is a finite specification of the solution of the problem given as an explicit combination of the primitives of the given tool in the specific order.
- What is the algorithm of the Turing machine?

	q0	q1	q2	...	h
s1	q1, s2, D	q3, s2, N	...	...	...
s2	q1, s1, D	q1, s2, L	...	...	...
s3	q2, s3, L	...	h, s3, N	...	...
...	...	...	...	...	...

Definitivness of an algorithm

- ♦ Is there a set of problems that cannot be solved with Turing machine
 - ♦ Is there a set of problems that cannot be solved with a digital computer?

DEFINITION:

Let us assume that we can connect all the computers in the world in one single supercomputer and let that computer work on a single problem. If that computer eventually finds a solution to the problem in the unlimited time (until the end of the universe), we still consider that problem “solvable”.

Definitivness of an algorithm

DEFINITION:

Let us assume that we can connect all the computers in the world in one single supercomputer and let that computer work on a single problem. If that computer eventually finds a solution to the problem in the unlimited time (until the end of the universe), we still consider that problem “solvable”.

- ◆ Even according to the above definition, it can be shown that there is a whole set of problems that cannot be solved neither with the Turing machine nor with the modern digital computer!

Definitivness and Turing machine

- ♦ Turing machine maps input information **A** to the output information **B**

CASE 1:

- ♦ After finite number of steps, machine reaches the halting state **h** and stops

CASE 2:

- ♦ Machine never reaches halting state
 - ♦ Turing machine is not applicable to the input information **A**

Algorithm complexity

- ♦ The analysis of algorithm complexity is the determination of the amount of resources necessary to execute the algorithm.
- ♦ Time complexity: quantifies the amount of time taken by an algorithm to run as a function of the size of the input data set
- ♦ Space complexity: quantifies the amount of memory space required for the algorithm as a function of the size of the input data set

Time complexity

- ♦ How can we access the time complexity of an algorithm?
 - ♦ By measuring time needed for the algorithm to execute with a particular data set?

QUESTIONS:

What execution time can we expect with the different data set?

What execution time can we expect on a different computer?

Time complexity

- ♦ How can we access the time complexity of an algorithm?
 - ♦ By measuring time needed for the algorithm to execute with a particular data set?

QUESTIONS:

What execution time can we expect with the different data set?

What execution time can we expect on a different computer?

TASK 2:

***Sort numbers:
3, 9, 8, 1, 15, 7, 10, 11, 6
in ascending order***

TASK 3:

***Write down precise steps you took
to sort the aforesaid sequence of
numbers***

TASK 4:

Write down precise steps necessary to sort a sequence of arbitrary length (length is known at the beginning of the execution)

TASK 5:

***Evaluate successful algorithms
and select the best one***

Bubble sort

```
function A=bubbleSort (A)
    n = length(A);           % Length of a sequence
    for i = n:-1:1           % From last to first number
        for j=1:i-1         % From first to (i-1)th
            if A(j) > A(j+1) % If next one is smaller
                tmp = A(j); % swap consecutive numbers
                A(j) = A(j+1);
                A(j+1) = tmp;
            end
        end
    end
end                          % Sequence is sorted
```

```

function A=bubbleSort (A)
    n = length(A);           % Length of a sequence
    for i = n:-1:1           % From last to first number
        for j=1:i-1          % From first to (i-1)th
            if A(j) > A(j+1) % If next one is smaller
                tmp = A(j);   % swap consecutive numbers
                A(j) = A(j+1);
                A(j+1) = tmp;
            end
        end
    end
end                          % Sequence is sorted

```

6 5 3 1 8 7 2 4

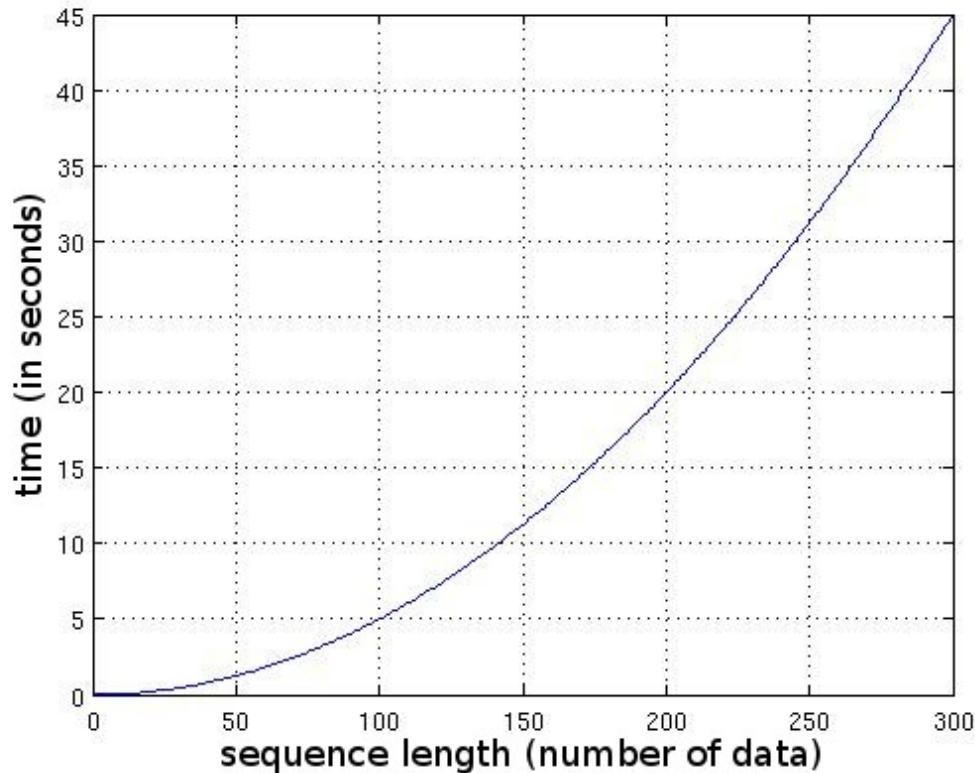
Bubble sort

If execution time of the sorting algorithm on the sequence of 100 numbers is 5 seconds, how long should it take to sort a sequence of 200 numbers?

Bubble sort

```
function A=bubbleSort (A)
    n = length(A);           % Number of operations:
    for i = n:-1:1           % n
        for j=1:i-1         % (n-1)+(n-2)+...+1=n(n-1)/2
            if A(j) > A(j+1)
                tmp = A(j);  % number of swap operations
                A(j) = A(i);  % depends on criterion
                A(i) = tmp;   % A(j)>A(j+1)
            end
        end
    End                       % Overall num. of operations
end                           % is proportional to n2
```

Time complexity of Bubble sort



- ▶ Estimated execution time, assuming that sorting of **100** numbers takes **5** sec.
- ▶ Estimated time for sorting **200** numbers is **20** sec, (**4** times longer)
- ▶ Estimated time for sorting **300** numbers is **45** seconds (**9** times longer)!

Time complexity

- Time complexity of an algorithm is defined as number of primitives (arithmetic, logic, memory access operations, ...) as a function of the size of the input data set

$$O(f(n))$$

n - number of input data
 $f(x)$ - complexity function

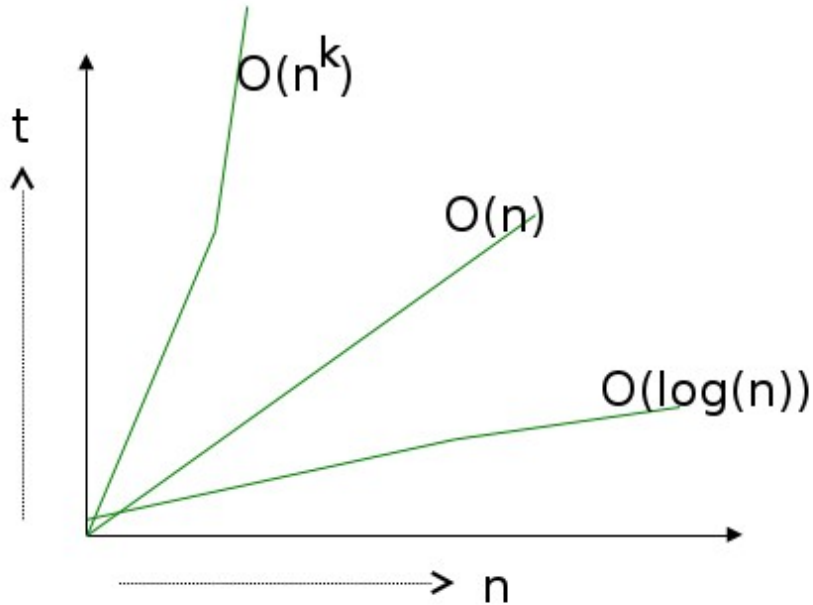
- Assessment of the time complexity is basically counting the number of operations.
- This approach gives estimate of the algorithm efficiency regardless of the implementation details and the available hardware.

Classes of Complexity

- Constant time: $O(1)$
- Logarithmic complexity: $O(\log(n))$
- Linear complexity: $O(n)$
- Polynomial time: $O(n^k)$
 - quadratic time $O(n^2)$, cubic time $O(n^3)$, ...
- Factorial time: $O(n!)$

• • •

Classes of Complexity



Efficiency of an algorithm is given with its complexity class:

Algorithms with complexities $O(n)$ and $O(3n)$ belong to the same complexity class

Efficiency of an algorithm with polynomial complexity is defined by exponent k (quadratic complexity, cubic complexity, ...)

NP complexity

- ♦ **NP** refers to “nondeterministic polynomial time”
 - ♦ One of the most fundamental complexity classes!
- “Any given solution to NP complex problem can be verified quickly, however there is no known efficient way to find a solution”
- ♦ The time required to solve an NP problem using any currently known algorithm increases very quickly as the size of the problem grows